Module 14 : Capstone Project: End-to-End Production GenAI Application

---

Build an AI-Powered Enterprise Knowledge Assistant — a full-stack application

Capstone Project: Build an AI-Powered Enterprise Knowledge Assistant — a full-stack application integrating:

(1) Advanced RAG with hybrid vector + graph retrieval and re-ranking over a multi-format document corpus

(2) Agentic tool use with LangGraph — code execution, web search, database queries, and MCP-based integrations

(3) Multi-layered guardrails — input validation, PII detection, hallucination checking, and output filtering

(4) Production infrastructure — FastAPI serving, Langfuse observability, semantic caching, and model routing

Deliverables: working application, architecture document, Promptfoo test suite (30+ cases), RAGAS evaluation report, safety scorecard

NOTE: Dummy data or data from online sources can be used

Real-World Business Scenario

You have joined the AI Platform Engineering Team at a large enterprise operating across multiple domains such as engineering, operations, compliance, and customer support.

The organization has thousands of internal documents (you can use dummy data):

- Policy documents (PDFs)
- Technical manuals
- SOPs and incident reports
- Knowledge base articles
- Structured data from internal databases
- Entity relationships across systems

Leadership Challenges

- Employees struggle to find accurate information quickly
- Knowledge is scattered across disconnected systems
- Existing search systems fail on complex, multi-hop queries
- High dependency on internal experts slows decision-making
- No mechanism to validate AI-generated responses for safety and compliance
- Lack of observability and evaluation in AI systems

Objective

Design and build a production-grade AI-powered knowledge assistant that:

- Retrieves accurate information from enterprise data
- Combines vector + graph retrieval
- Uses agent-based reasoning and tool execution
- Applies multi-layered guardrails
- Provides observability, evaluation, and performance monitoring

Dataset and Data Sources

You will work with a multi-format enterprise dataset:

Unstructured Data

- PDF documents (policies, manuals)
- Markdown or text files
- FAQ knowledge base

Structured Data

- CSV / SQL tables (products, users, transactions)
- Metadata tables

Graph Data

- Entities (people, systems, components)
- Relationships (depends_on, owns, related_to)

Core Business Challenges to Solve

1. Knowledge Retrieval Challenges

- Users cannot find relevant information quickly
- Vector search alone fails for multi-hop reasoning

2. Context Understanding Problems

- Answers lack grounding and citations
- Long documents are not effectively utilized

3. Decision-Making Delays

- Employees rely on manual searches or experts
- No unified intelligent assistant

4. Safety and Compliance Risks

- Potential exposure of sensitive data (PII)
- Hallucinated or unsafe outputs

5. Lack of Observability

- No visibility into model behavior, latency, or failures

Requirements Gathering

Stakeholder Mapping and Roles

Stakeholder	Requirement
CTO	Scalable, production-ready architecture
Engineering Team	Accurate technical answers with citations
Operations Team	Quick SOP retrieval
Compliance Team	Safe and policy-compliant outputs
AI Platform Team	Monitoring, evaluation, optimization
End Users	Fast, reliable responses

Functional Requirements

- Natural language Q&A system
- Hybrid retrieval (vector + graph)
- Re-ranking for improved accuracy
- Agent-based tool execution
- Source citations in responses
- Multi-step reasoning capability
- Session-based interaction

Non-Functional Requirements

- Low latency responses
- High accuracy and faithfulness
- Scalable API-based system

- Observability with tracing
- Secure handling of sensitive data
- Cost-efficient model usage

Data Requirements

- Clean and structured document corpus
- Metadata for filtering and ranking
- Entity-relation extraction for graph
- Evaluation dataset (test queries)

Scope Definition

In Scope

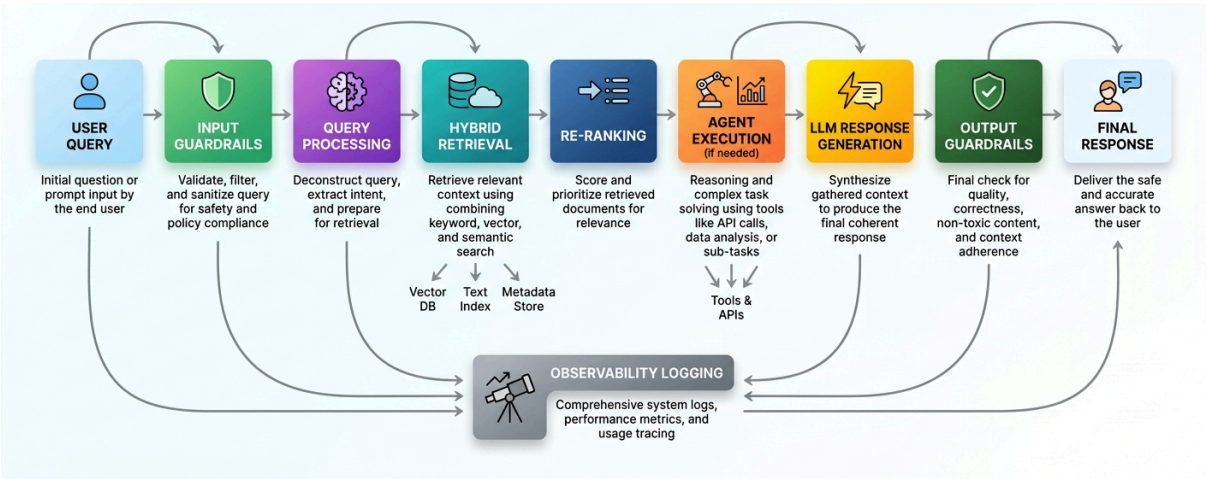
- Hybrid RAG system (vector + graph)
- LangGraph agent with tools
- Guardrails implementation
- FastAPI backend
- Langfuse observability
- Promptfoo test suite (30+ cases)
- RAGAS evaluation
- Safety scorecard

Out of Scope

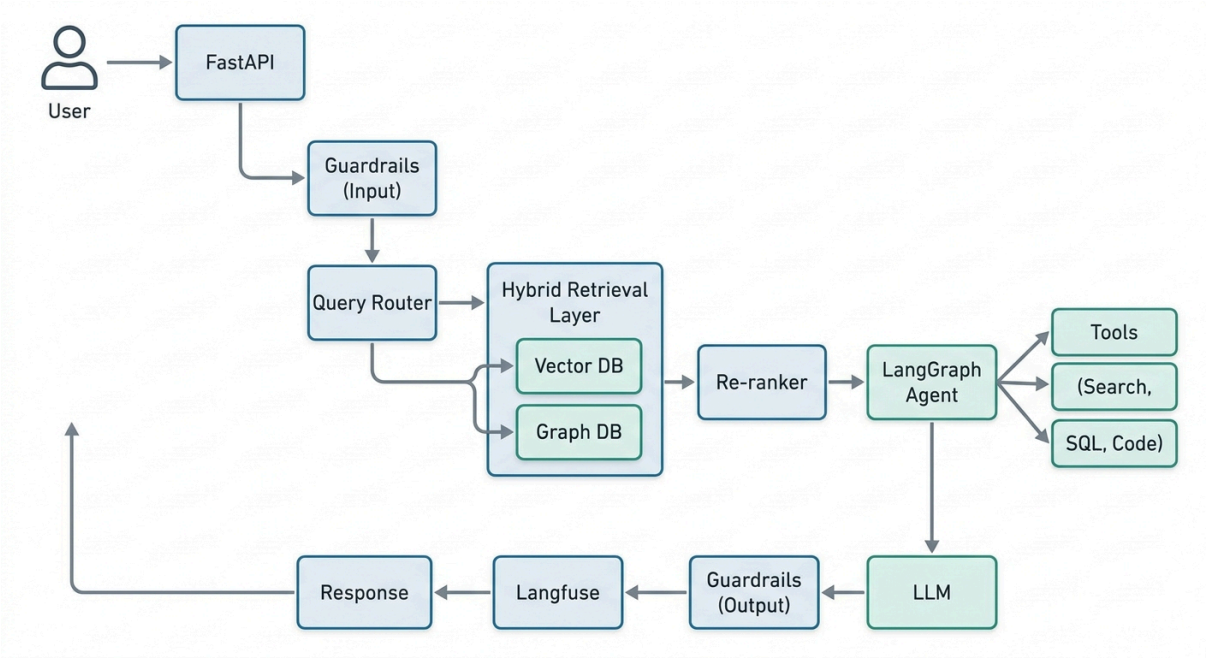
- Full frontend application
- Enterprise authentication systems
- Distributed large-scale infrastructure
- Model pre-training

End-to-End Implementation Blueprint

System Workflow



Architecture Diagram (for AI generation tools)



Implementation Tasks

TASK 0 – Project Setup

Goal: Set up project structure and environment

Steps:

- Create virtual environment
- Install dependencies
- Configure .env
- Set up project folders

TASK 1 – Data Ingestion and Preprocessing

Goal: Prepare document corpus

Steps:

- Load PDFs and text files
- Clean and normalize text
- Extract metadata
- Store processed documents

TASK 2 – Vector Indexing

Goal: Enable semantic retrieval

Steps:

- Chunk documents
- Generate embeddings
- Store in vector database (ChromaDB)
- Validate retrieval

TASK 3 – Knowledge Graph Construction

Goal: Enable multi-hop reasoning

Steps:

- Extract entities and relations
- Build graph in Neo4j
- Validate queries

TASK 4 – Hybrid Retrieval System

Goal: Combine vector and graph retrieval

Steps:

- Perform vector search
- Perform graph traversal
- Merge results
- Apply re-ranking

TASK 5 – Agent Implementation (LangGraph)

Goal: Enable tool-based reasoning

Tools to include:

- Web search
- Query tool
- Python execution

Steps:

- Define agent workflow
- Add tool selection logic
- Handle multi-step reasoning

TASK 6 – Guardrails Implementation

Goal: Ensure safe AI responses

Steps:

- Input validation
- Prompt injection detection
- PII detection and masking
- Hallucination checks
- Output filtering

TASK 7 – FastAPI Backend

Goal: Expose production API

Steps:

- Create /chat endpoint
- Handle user sessions
- Integrate pipeline
- Return structured responses

TASK 8 – Observability with Langfuse

Goal: Track system performance

Steps:

- Add tracing
- Track latency
- Track token usage
- Monitor errors

TASK 9 – Prompt Testing (Promptfoo)

Goal: Automate evaluation

Steps:

- Create 30+ test cases
- Define expected outputs
- Run regression tests
- Analyze failures

TASK 10 – RAGAS Evaluation

Goal: Measure retrieval quality

Metrics:

- Faithfulness
- Answer relevance
- Context precision
- Context recall

TASK 11 – Safety Evaluation

Goal: Test guardrails

Steps:

- Run adversarial prompts
- Measure failures
- Create safety scorecard

TASK 12 – Performance and Load Testing

Goal: Ensure production readiness

Steps:

- Measure latency

- Test caching
- Evaluate throughput

TASK 13 – Deployment and Documentation

Goal: Finalize project

Steps:

- Organize project
- Document architecture
- Prepare final outputs

Deliverables

You must submit:

- Designed Application - Github Repository Link
- Logged Documentation - Overview, Architecture, Solution Approach, Stepwise Solution
- Hybrid RAG implementation
- LangGraph agent workflow
- Guardrails implementation
- Langfuse tracing screenshots
- Promptfoo test suite
- RAGAS evaluation report
- Safety scorecard

Real-world Evaluation Rubric

Criteria

Architecture Design
Retrieval Quality
Agent Workflow
Guardrails Implementation
Testing & Evaluation
Production Readiness
Documentation

Final Documentation & Walkthrough Summary

Prepare a walkthrough covering:

1. Business problem
2. System architecture
3. Retrieval strategy (vector + graph)
4. Agent workflow
5. Guardrails implementation
6. Evaluation results
7. Observability insights
8. Challenges and improvements

Expected Outcomes

- A fully integrated production-style GenAI system
- Strong understanding of real-world AI architecture
- Hands-on experience with evaluation and safety
- Portfolio-ready project

Summary

- This capstone integrates all modules into one system
- Combines RAG, agents, guardrails, evaluation, and deployment
- Mimics real enterprise AI engineering workflows
- Prepares learners for roles like AI Engineer and LLM Ops